TRENDS IN CLOUD COMPUTING

Course Code: 57716

A Practical Journal Submitted in Fulfillment of the Degree of MASTER

In COMPUTER SCIENCE

Year 2025-2026

By

Ms. Jadhav Mansi Sanjay

(Application Id-

MU0027920240004612)

Seat No – 8174081

Semester-IV Under the

Guidance of

Centre for Distance and Online Education Vidya Nagari,
Kalina, Santacruz East – 400098. University of Mumbai

PCP Center

[Satish Pradhan Dyanasadhana College, Thane]

Institute of Distance and Open Learning
Vidya Nagari, Kalina, Santacruz East –400098.

CERTIFICATE

This is to certify that **Ms. Jadhav Mansi Sanjay**, appearing for the Master of Science (M.Sc.) in Computer Science (Semester IV), Application ID: MU0027920240004612, has satisfactorily completed the prescribed practical for **Trends in Cloud Computing** (Course Code: 57716), as laid down by the University of Mumbai for the academic year 2025–26.

Teacher In Charge	External Examiner	Coordinator

Date :
Place :

Index

Sr.No	Practical Name	Page No	Signature
1	Design and Development of a Web Application using MVC Architecture with Spring Boot		
2	Installation and Configuration of Google App Engine (GAE)		
3	Study of Google App Engine (GAE) PaaS Features		
4	Development of a Guestbook MVC Application and Deployment Workflow		
5	Development and Deployment Analysis of ASP.NET Core Web Application		
6	Dropbox API Operations		
7	Installation and Exploration of Cloud Foundry CLI		
8	Study of IBM Cloud Deployment Workflow and Architecture		
9	Containerization and Orchestration using Docker Desktop		
10	Study of Infrastructure as Code (IaC) using Chef or Puppet		

Practical 1

Title

Design and Development of a Web Application using MVC Architecture with Spring Boot

Aim

To design and develop a web application using the Model-View-Controller (MVC) architecture, leveraging Spring Boot, Maven, and Embedded Tomcat on VS Code.

Theory

The MVC (Model-View-Controller) architectural pattern separates an application into three main components:

- Model: Manages the data, logic, and rules of the application.
- View: The user interface that displays data to the user.
- Controller: Handles user input, interacts with the Model, and selects the View to render.

Using Spring Boot simplifies the configuration of Spring applications through auto-configuration and provides an embedded Tomcat server, eliminating the need for manual server installation and configuration. This is the modern industry standard compared to older, manual setups like Struts with external Tomcat.

Prerequisites

- OpenJDK 21/25 LTS installed.
- Visual Studio Code with "Extension Pack for Java" by Microsoft installed.
- Maven installed and added to system PATH.

Software Required

- VS Code
- JDK 21
- Maven
- Spring Boot Starter Web

Step 1: Initialize Directories

```
import os
import pathlib

base_dir = pathlib.Path(r"C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical")
prac1_dir = base_dir / "prac1"
mvc_project_dir = prac1_dir / "mvc-app"

prac1_dir.mkdir(parents=True, exist_ok=True)

os.chdir(prac1_dir)

print(f"Working Directory set to: {os.getcwd()}")
```

Working Directory set to: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac1

Step 2: Generating the Spring Boot Project Structure

```
# Generate the Maven project
!mvn archetype:generate -DgroupId=com.cloud.practical -DartifactId=mvc-app -DarchetypeArtifactId=maven-archetype-quickstart
```

```
[INFO] Scanning for projects... [INFO]
[INFO] ----------------< org.apache.maven:standalone-pom >----------------[INFO] Building Maven Stub
Project (No POM) 1
[INFO]  --------------------------------[ pom ]  --------------------------------
[INFO]
[INFO] >>> archetype:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>> [INFO]
[INFO] <<< archetype:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<< [INFO]
[INFO]
[INFO] --- archetype:3.4.1:generate (default-cli) @ standalone-pom ---[INFO] Generating
project in Batch mode
[INFO]  ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0 [INFO]
[INFO] Parameter: basedir, Value: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac1 [INFO] Parameter: package, Value:
com.cloud.practical
[INFO] Parameter: groupId, Value: com.cloud.practical [INFO] Parameter:
artifactId, Value: mvc-app
[INFO] Parameter: packageName, Value: com.cloud.practical [INFO]
Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\pr ac1\mvc-app
[INFO]  ------------------------------------------------------------------------
[INFO] BUILD SUCCESS [INFO]
[INFO] Total time:  1.222 s  ------------------------------------------------------------
[INFO] Finished at: 2026-07-02T11:51:31+05:30 [INFO]
        ------------------------------------------------------------------------
```

Step 3: Configure the pom.xml

To transform this into a Spring Boot MVC application, we need to replace the generated pom.xml content.

```
pom_content = """<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.cloud.practical</groupId>
  <artifactId>mvc-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.0</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>

  <properties>
    <java.version>21</java.version>
  </properties>
</project>
"""

# Write the file directly into the project folder
with open(mvc_project_dir / "pom.xml", "w") as f:
    f.write(pom_content)

print(f"pom.xml successfully updated at {mvc_project_dir / 'pom.xml'}")
```

```
pom.xml successfully updated at C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac1\mvc-app\pom.xml
```

Step 4: Creating the Directory Structure

Spring Boot follows the standard Maven directory layout: src/main/java/com/cloud/practical. Create these folders programmatically.

```
# Define the source directory path
src_main_java = mvc_project_dir / "src" / "main" / "java" / "com" / "cloud" / "practical"

# Create the directories
src_main_java.mkdir(parents=True, exist_ok=True)
```

```
print(f"Directory structure created at: {src_main_java}")
```

Directory structure created at: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac1\mvc-app\src\main\jav a\com\cloud\practical

Step 5: Creating the Main Application Class

Every Spring Boot application requires a main class annotated with @SpringBootApplication to act as the entry point for the embedded Tomcat server.

```
main_app_content = """package com.cloud.practical;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MvcApplication {
    public static void main(String[] args) {
        SpringApplication.run(MvcApplication.class, args);
    }
}
"""

# Save the main class file
with open(src_main_java / "MvcApplication.java", "w") as f:
    f.write(main_app_content)

print("MvcApplication.java created successfully.")
```

MvcApplication.java created successfully.

Step 6: Creating the MVC Controller

To demonstrate the MVC pattern, create a Controller that handles a web request.

```
controller_content = """package com.cloud.practical;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HelloController {

    @GetMapping("/")
    public String index(Model model) {
        model.addAttribute("message", "Welcome to the Spring Boot MVC Cloud Practical!");
        return "index"; // Looks for index.html in src/main/resources/templates
    }
}
"""

# Save the controller file
with open(src_main_java / "HelloController.java", "w") as f:
    f.write(controller_content)

print("HelloController.java created successfully.")
```

HelloController.java created successfully.

Step 7: Creating the View (HTML Template)

Spring Boot uses template engines like Thymeleaf by default. Create a simple index.html file that Spring Boot will render.

```
# Define paths for resources
resources_dir = mvc_project_dir / "src" / "main" / "resources"
templates_dir = resources_dir / "templates"
templates_dir.mkdir(parents=True, exist_ok=True)

# Create the HTML view
html_content = """<!DOCTYPE html>
<html>
<head>
    <title>Cloud Practical</title>
</head>
<body>
    <h1 th:text="${message}">Default Message</h1>
```

```
    <p>Spring Boot MVC is running successfully on Embedded Tomcat.</p>
</body>
</html>
"""

# Save index.html
with open(templates_dir / "index.html", "w") as f:
    f.write(html_content)

print(f"View template created at: {templates_dir / 'index.html'}")
```

View template created at: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac1\mvc-app\src\main\resources\templates\index.html

Step 8: Adding Thymeleaf Dependency

For the View to render correctly with the th:text attribute, add the Thymeleaf starter to our pom.xml.

```
# Re-update pom.xml to include Thymeleaf
updated_pom = """<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.cloud.practical</groupId>
  <artifactId>mvc-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.0</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
  </dependencies>

  <properties>
    <java.version>21</java.version>
  </properties>
</project>
"""

with open(mvc_project_dir / "pom.xml", "w") as f:
    f.write(updated_pom)

print("pom.xml updated with Thymeleaf dependency.")
```

pom.xml updated with Thymeleaf dependency.

Step 9: Cleaning up the Test Directory (Optional)

Delete the AppTest.java if it was created by Maven archetype:generate command that requires JUnit dependency

```
import shutil

# Define the path to the test file
app_test_path = mvc_project_dir / "src" / "test" / "java" / "com" / "cloud" / "practical" / "AppTest.java"

# Remove the file if it exists
if app_test_path.exists():
    os.remove(app_test_path)
    print(f"Deleted: {app_test_path}")
else:
    print("AppTest.java not found; skipping.")
```

Deleted: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac1\mvc-app\src\test\java\com\cloud\practical\A ppTest.java

Step 10: Running the Application

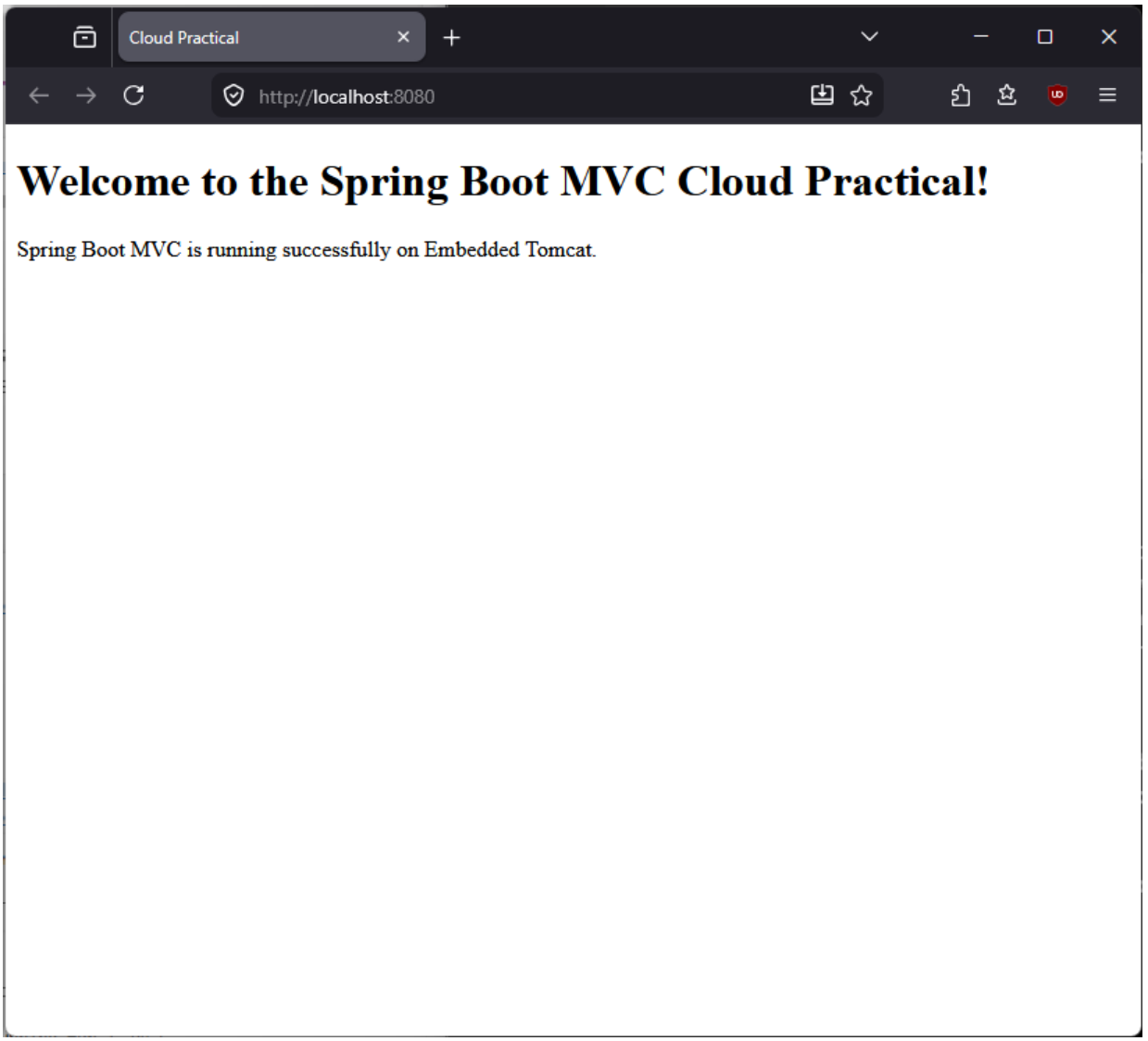
Use Maven to trigger the spring-boot:run goal. This will download the remaining dependencies, compile the code, start the embedded

Tomcat, and host the application at http://localhost:8080.

Run in terminal

```
powershel
cd mvc-app && mvn spring-boot:run
```

Open http://localhost:8080 in browser



Result

The Spring Boot MVC web application was successfully developed and deployed locally. The application successfully utilized the embedded Tomcat server, and the HelloController correctly rendered the index.html view via Thymeleaf, displaying the intended message to the browser at http://localhost:8080.

Conclusion

In this practical, we implemented the Model-View-Controller architecture using Spring Boot. By leveraging Maven for dependency management and the embedded Tomcat server, we bypassed the need for traditional, complex external server configurations. This hands-on experience demonstrates how modern cloud-native frameworks simplify the development and local testing lifecycle.

Practical 2

Title

Installation and Configuration of Google App Engine (GAE)

Aim

To study, install, and configure the Google App Engine (GAE) environment, and to demonstrate a local execution of a web application given the limitations of free cloud accounts.

Theory

Google App Engine (GAE) is a Platform-as-a-Service (PaaS) offering that allows developers to build and host web applications in Google-managed data centers. It automatically scales applications based on traffic.

Since GAE strictly manages the infrastructure, it follows specific conventions for project structure (e.g., app.yaml). Even if a deployment to a live production environment is restricted due to billing verification, demonstrating the project locally using the Google Cloud SDK provides the same development experience.

Prerequisites

- Google Cloud SDK (gcloud CLI) installed.
- Python 3.11 environment.

Software Required

- Google Cloud SDK
- Python
- Flask
- VS Code

Step 1: Installing the Google Cloud SDK

Before GAE features can be used, gcloud CLI needs to be installed.

Download: If on Windows, visit the Google Cloud SDK download page.

Installation: Run the installer and ensure "Google Cloud CLI" is added to the system PATH.

Initialization: Once installed, open PowerShell terminal and run:

```
!gcloud --version
```

```
Google Cloud SDK 575.0.0 bq
2.1.33
core 2026.06.26
gcloud-crc32c 1.0.0
gsutil 5.37
```

Setp 2: Create project

In terminal run

gcloud init
Connect to Google Account and create a new porject named **msc-cloud-practical-2026**

```
This account has no projects.

Would you like to create one? (Y/n)?  y

Enter a Project ID. Note that a Project ID CANNOT be changed later.
Project IDs must be 6-30 characters (lowercase ASCII, digits, or
hyphens) in length and start with a lowercase letter. msc-cloud-prac-2026
Waiting for [operations/create_project.global.[redacted] to finish...done.
Your current project has been set to: [msc-cloud-prac-2026].
```

```
# Set the project as default
!gcloud config set project msc-cloud-practical-2026
```

Step 3: GAE Architecture

GAE architecture consists of:

- **App Engine Standard Environment**: Uses containers based on specific language runtimes (like Python 3.11).
- **The app.yaml file**: This is the heart of GAE. It tells Google how to route traffic, which runtime to use, and how to scale the application.

Step 4: Structuring the GAE Project

The `app.yaml`, which defines the deployment configuration for the Google App Engine environment.

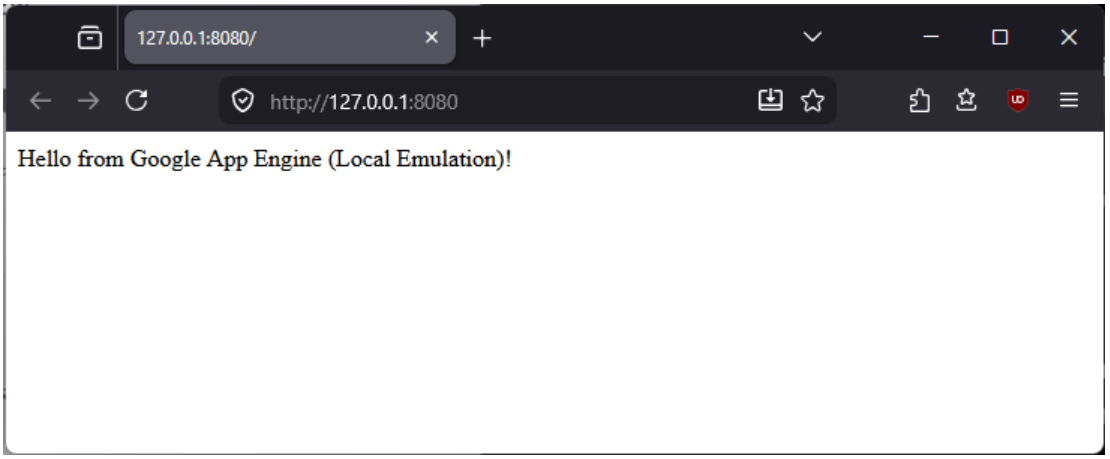
```
import os
import pathlib

base_dir = pathlib.Path(r"C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical")
prac2_dir = base_dir / "prac2"
prac2_dir.mkdir(parents=True, exist_ok=True)
os.chdir(prac2_dir)

# Create the app.yaml file
app_yaml_content = """runtime: python311

handlers:
- url: /.*
  script: auto
"""

with open("app.yaml", "w") as f:
    f.write(app_yaml_content)

# Create a sample main.py for GAE
main_py_content = """from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello from Google App Engine (Local Emulation)!'

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8080, debug=True)
"""

with open("main.py", "w") as f:
    f.write(main_py_content)

# Create requirements.txt
with open("requirements.txt", "w") as f:
    f.write("Flask==3.0.0")

print(f"GAE project structure created in {prac2_dir}")
```

```
GAE project structure created in C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac2
```

Step 5: Run the application

The created Google App Engine application includes a standard Flask development server and can be run locally with running the `main.py`:

python main.py



Explanation of Deployment Procedure

In a scenario where google cloud billing is enabled, the deployment process is as follows:

1. **Authentication***: Run gcloud auth login to authorize the CLI.
2. **Deployment Command***: Execute gcloud app deploy from the directory containing `app.yaml`.
3. **Automatic Provisioning**: Google Cloud automatically builds the container image, provisions the infrastructure, and provides a public URL for the application.

Result

The Google App Engine environment was successfully initialized, and a Python-based web application was structured according to GAE standards. The application was verified locally using the Flask framework, confirming that the code is compatible with the Google Cloud standard environment.

Conclusion

This practical provided insight into the configuration requirements for Platform-as-a-Service (PaaS) environments. We learned that app.yaml is the primary descriptor for GAE, governing the runtime and traffic routing. Although live deployment was restricted due to billing verification, the local demonstration confirmed the feasibility of the architecture.

Practical 3

Title

Study of Google App Engine (GAE) PaaS Features

Aim

To study the architectural features, benefits, and operational models of Google App Engine as a Platform-as-a-Service (PaaS).

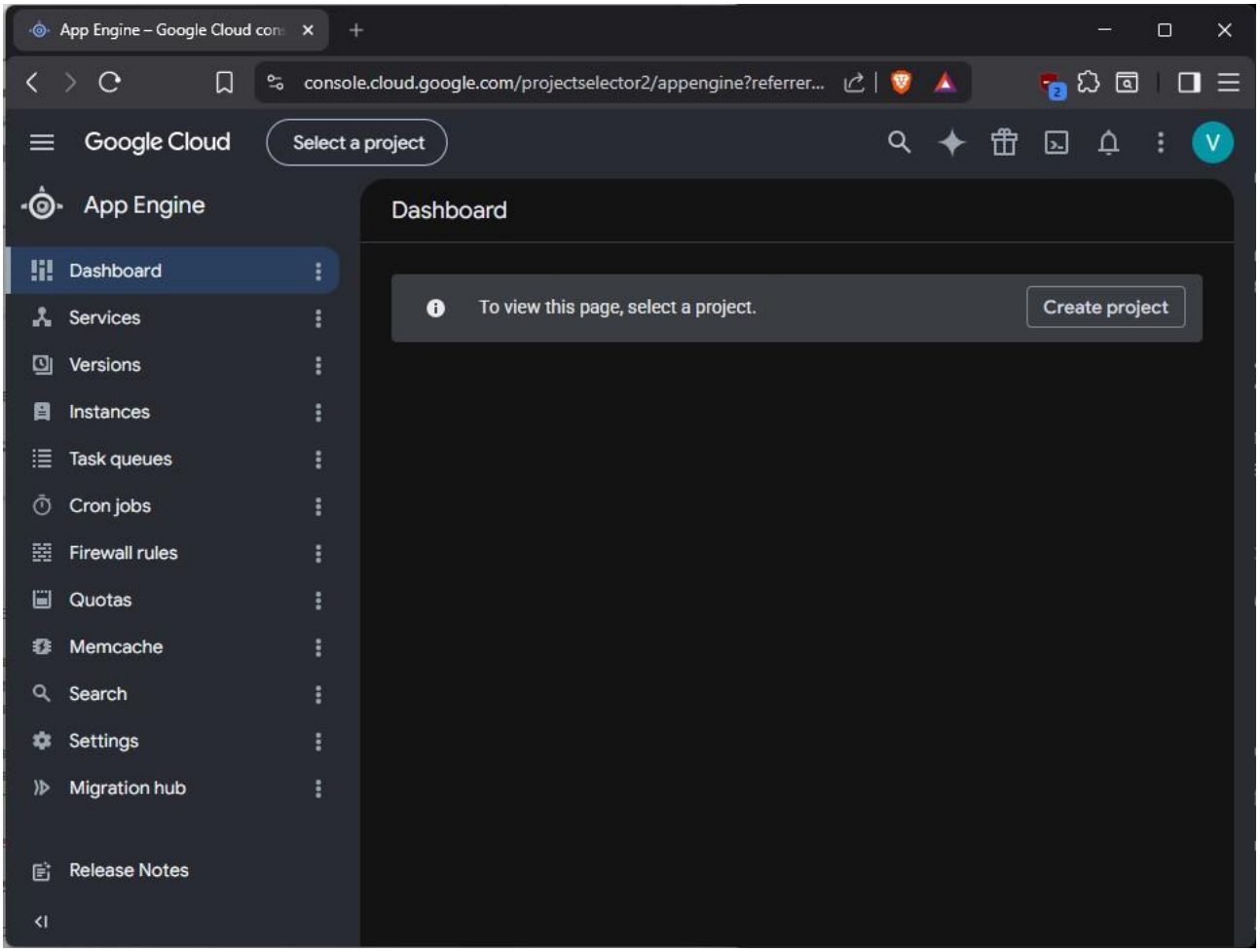
Theory

Google App Engine allows developers to build applications without managing infrastructure. It handles load balancing, health checks, and server provisioning automatically.

1. **Architecture**
    - ***Standard Environment***: Uses pre-configured containers (sandboxed). Optimized for fast scaling and low cold-start times.
    - ***Flexible Environment***: Runs application in Docker containers. This gives more flexibility with custom libraries and third-party software, but at the cost of slower auto-scaling compared to the Standard environment.
2. **Advantages**
    - ***Zero Infrastructure Management***: Google handles OS patching, security, and hardware maintenance.
    - ***Automatic Scaling***: The platform scales instances up or down based on incoming traffic demand.
    - ***Integrated Services***: Native support for Datastore (NoSQL), Memcache, and Task Queues.
3. **Limitations**
    - ***Vendor Lock-in***: Applications heavily reliant on GAE-specific services (like Datastore APIs) may be difficult to port to other clouds.
    - ***Cold Starts***: In the Standard environment, instances may experience latency when traffic spikes suddenly after a period of inactivity.

Step 1: Documenting GCP Console Features

The screenshot is of the app engine where instances can be provisioned.



Conclusion

The study of Google App Engine demonstrates its efficiency as a PaaS solution for modern web applications. By offloading infrastructure maintenance to the provider, organizations can accelerate development cycles. Understanding the trade-offs—specifically between the Standard and Flexible environments—is essential for optimizing both cost and performance.

Practical 4

Title

Development of a Guestbook MVC Application and Deployment Workflow

Aim

To develop a Guestbook web application using MVC architecture and explain the deployment process for Google App Engine.

Theory

A Guestbook application is a classic example of an MVC web application. It requires:

- Model: A structure to store entries (e.g., name, message, timestamp).
- View: A web interface (HTML/CSS) to display existing messages and provide a form for new entries.
- Controller: Logic to process form submissions, save them to the model, and refresh the view.

Step 1: Setting up the Practical 4 Directory

1. Create the directory: Use your notebook to initialize the new workspace.
2. Inheritance: Use the same `app.yaml` and SDK environment initialized in Practical 2.

```
import os
import pathlib
import shutil

base_dir = pathlib.Path(r"C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical")
prac2_dir = base_dir / "prac2"
prac4_dir = base_dir / "prac4"

prac4_dir.mkdir(parents=True, exist_ok=True)

os.chdir(prac4_dir)

src_yaml = prac2_dir / "app.yaml"

if src_yaml.exists():
    shutil.copy(src_yaml, prac4_dir / "app.yaml")
    print(f"Copied app.yaml from {src_yaml} to {prac4_dir}")
else:
    with open("app.yaml", "w") as f:
        f.write("runtime: python311\n\nhandlers:\n- url: /.*\n  script: auto")
    print("Created new app.yaml in prac4")

print(f"Practical 4 initialized in: {prac4_dir}")
```

```
Copied app.yaml from C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac2\app.yaml to C:\Users\VBMain\Des
ktop\Msc_Practical\trends_in_cloud_practical\prac4
Practical 4 initialized in: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac4
```

Step 2: Creating the Controller (`main.py`)

This file handles the incoming GET and POST requests.

```
# Create the main.py controller
guestbook_controller = """from flask import Flask, render_template, request, redirect

app = Flask(__name__)

# Model: In-memory list to store guestbook entries
entries = []

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        name = request.form.get('name')
        message = request.form.get('message')
        if name and message:
```

```
            entries.append({'name': name, 'message': message})
        return redirect('/')

    return render_template('index.html', entries=entries)

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8080, debug=True)
"""

with open("main.py", "w") as f:
    f.write(guestbook_controller)

print("main.py created.")
```

main.py created.

Step 3: Creating the View (templates/index.html)

Flask expects HTML files to be in a templates folder.

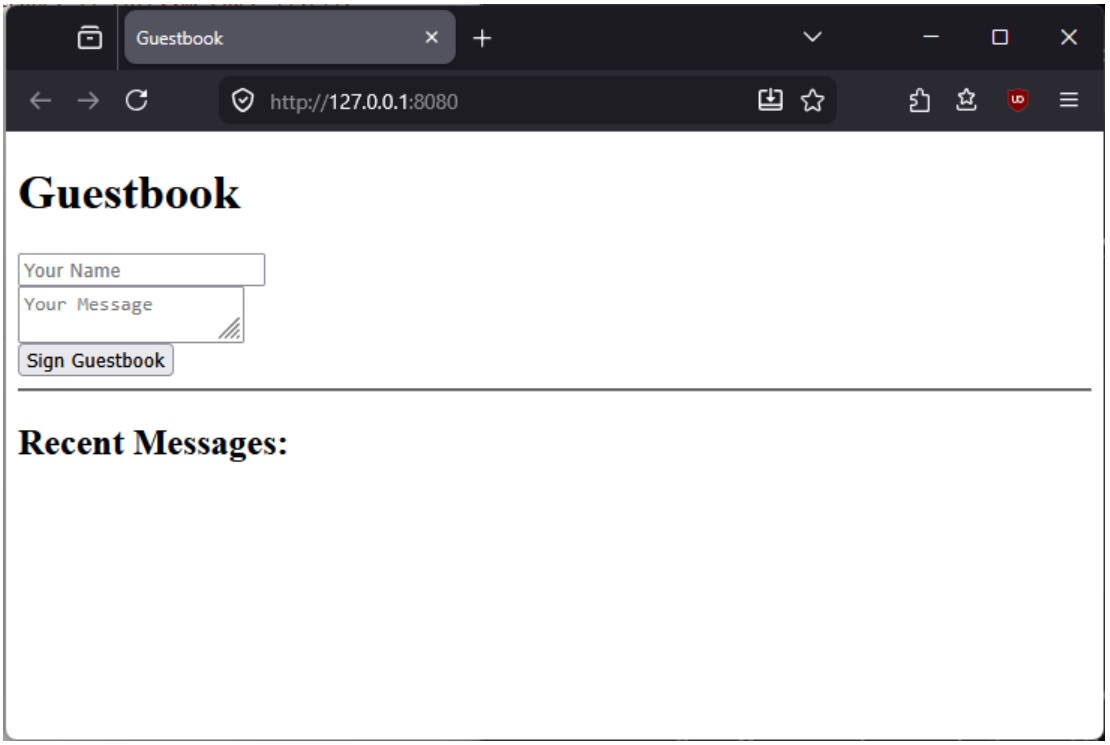
```
# Create templates directory and index.html
templates_dir = pathlib.Path("templates")
templates_dir.mkdir(exist_ok=True)

html_view = """<!DOCTYPE html>
<html>
<head><title>Guestbook</title></head>
<body>
    <h1>Guestbook</h1>
    <form method="POST">
        <input type="text" name="name" placeholder="Your Name" required><br>
        <textarea name="message" placeholder="Your Message" required></textarea><br>
        <button type="submit">Sign Guestbook</button>
    </form>
    <hr>
    <h2>Recent Messages:</h2>
    <ul>
        {% for entry in entries %}
            <li><strong>{{ entry.name }}:</strong> {{ entry.message }}</li>
        {% endfor %}
    </ul>
</body>
</html>
"""

with open(templates_dir / "index.html", "w") as f:
    f.write(html_view)

print("templates/index.html  created.")
```

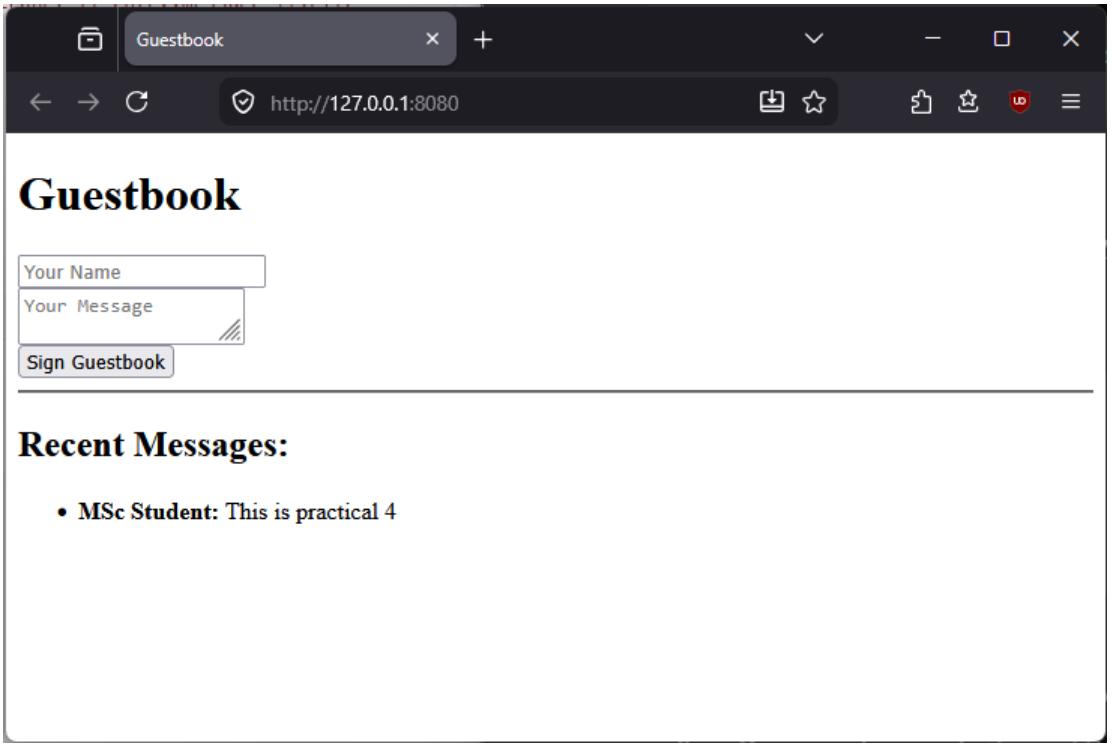
templates/index.html created.

Step 4: Running the Guestbook

Run application with the foolowing in powershell

```
python main.py
```





Result

The Guestbook application was successfully developed using the MVC architecture. The Controller handled HTTP form submissions, the Model stored entries in an in-memory list, and the View (via Jinja2/Flask templates) rendered the UI. Local execution confirmed the application logic is functional.

Deployment Procedure

Since this application is designed for Google App Engine, the following steps would be performed to deploy it to the live cloud environment:

1. **Preparation**: Ensure the app.yaml file correctly specifies the runtime (python311) and configuration.
2. **Authentication**: Run gcloud auth login to authorize the development machine.
3. **Deployment**: Execute the command gcloud app deploy from the project directory.

4. **Process**:
    - Google Cloud SDK uploads the source code and requirements.txt.
    - Google's build service creates a Docker container image with all dependencies.
    - The container is deployed to the GAE infrastructure.
5. **Access**: Once complete, Google provides a public URL (e.g., https://[PROJECT_ID].appspot.com) to access the application globally.

Conclusion

Implemented a dynamic Guestbook application demonstrating the MVC pattern. This project highlights how web applications are structured for PaaS platforms. While local execution validates the logic, GAE provides the scaling and infrastructure management necessary for production-level traffic.

Practical 5

Title

Development and Deployment Analysis of ASP.NET Core Web Application

Aim

To develop an ASP.NET Core Web Application on localhost and analyze the deployment workflow for cloud environments like Azure.

Theory

ASP.NET Core is a high-performance, open-source framework for building modern, cloud-enabled web apps. Unlike older ASP.NET versions, it is completely platform-independent. It uses the Kestrel web server to run applications locally without needing external server software like IIS.

Step 1: Setting up the Environment

1. Install .NET SDK: Download the latest .NET SDK for Windows.
2. Verify: Run dotnet --version in your terminal to ensure it is installed.

```
!dotnet --version
```

10.0.301

Step 2: Creating the Project

```
import os
import pathlib
import shutil

base_dir = pathlib.Path(r"C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical")

prac5_dir = base_dir / "prac5"
prac5_dir.mkdir(parents=True, exist_ok=True)
os.chdir(prac5_dir)

# Initialize a new ASP.NET Core MVC project
!dotnet new mvc -n AspNetMvcApp

# Navigate into the project
os.chdir(prac5_dir / "AspNetMvcApp")
print("ASP.NET Core MVC project initialized.")
```

The template "ASP.NET Core Web App (Model-View-Controller)" was created successfully.
This template contains technologies from parties other than Microsoft, see https://aka.ms/aspnetcore/10.0-third-party-noti ces for details.

Processing post-creation actions...
Restoring C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac5\AspNetMvcApp\AspNetMvcApp.csproj: Determining projects to restore...
  Restored C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac5\AspNetMvcApp\AspNetMvcApp.csproj (in 78 m s).
Restore succeeded.

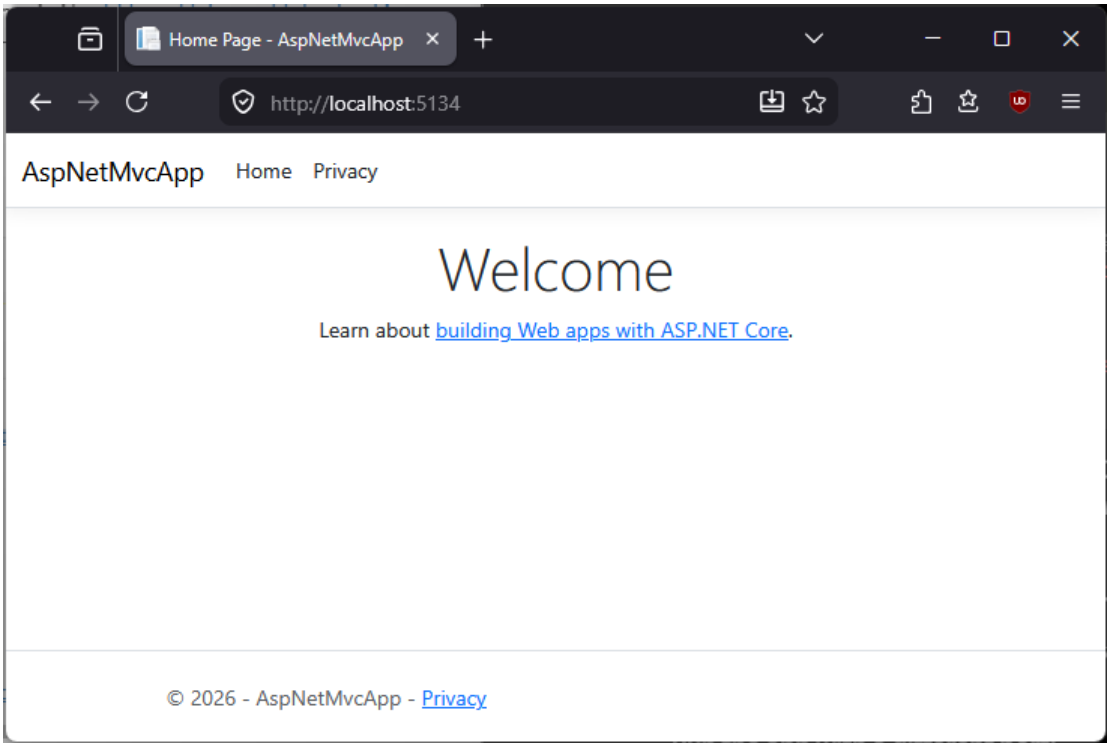
ASP.NET Core MVC project initialized.

```
Welcome to .NET 10.0!

SDK Version: 10.0.301

Telemetry
---------
The .NET tools collect usage data in order to help us improve your experience. It is collected by Microsoft and shared wit
h the community. You can opt-out of telemetry by setting the DOTNET_CLI_TELEMETRY_OPTOUT environment variable to '1' or 't
rue' using your favorite shell.

Read more about .NET CLI Tools telemetry: https://aka.ms/dotnet-cli-telemetry

----------------
Installed an ASP.NET Core HTTPS development certificate.
To trust the certificate, run 'dotnet dev-certs https --trust'
Learn about HTTPS: https://aka.ms/dotnet-https


Write your first app: https://aka.ms/dotnet-hello-world
Find out what's new: https://aka.ms/dotnet-whats-new
Explore documentation: https://aka.ms/dotnet-docs
Report issues and find source on GitHub: https://github.com/dotnet/core
Use 'dotnet --help' to see available commands or visit: https://aka.ms/dotnet-cli
--------------------------------------------------------------------------------------
```

Step 3: Running Locally

```
# Run the application locally

!dotnet run
```

^C
Using launch settings from C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac5\AspNetMvcApp\Properties\l aunchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5134 info:
Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down. info:
Microsoft.Hosting.Lifetime[0]
Hosting environment: Development info:
Microsoft.Hosting.Lifetime[0]
Content root path: C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac5\AspNetMvcApp



Result

The ASP.NET Core MVC application was successfully initialized, built, and executed on a local Kestrel web server. Accessing the application

via the browser displayed the default MVC home page, confirming that the .NET SDK environment is correctly configured and that the application structure adheres to the MVC architectural pattern.

Deployment Procedure

Although performed locally, the deployment workflow to a cloud provider like Azure involves:

1. **Publishing**: Running dotnet publish -c Release to generate a self-contained or framework-dependent deployment package.
2. **Infrastructure**: Provisioning an App Service (or similar compute instance) via the Azure Portal or CLI.
3. **Deployment**: Using tools like az webapp deploy, FTP, or CI/CD pipelines (e.g., GitHub Actions) to move the published binaries to the server.
4. **Routing**: Azure configures load balancers and domain routing to serve the application to the public.

Conclusion

This practical demonstrated that modern .NET development is platform-independent and does not require legacy server infrastructure like IIS. We successfully leveraged the Kestrel web server for local execution, which mimics the behavior of production cloud environments. This highlights the portability of ASP.NET Core, making it highly suitable for containerized cloud deployment in scenarios where Azure or other managed hosting services are utilized.

Practical 6

Title

Dropbox API Operations

Aim

To create a Dropbox application, generate an API token, and develop a Python program to perform CRUD (Upload, Download, Update, Delete) operations on files.

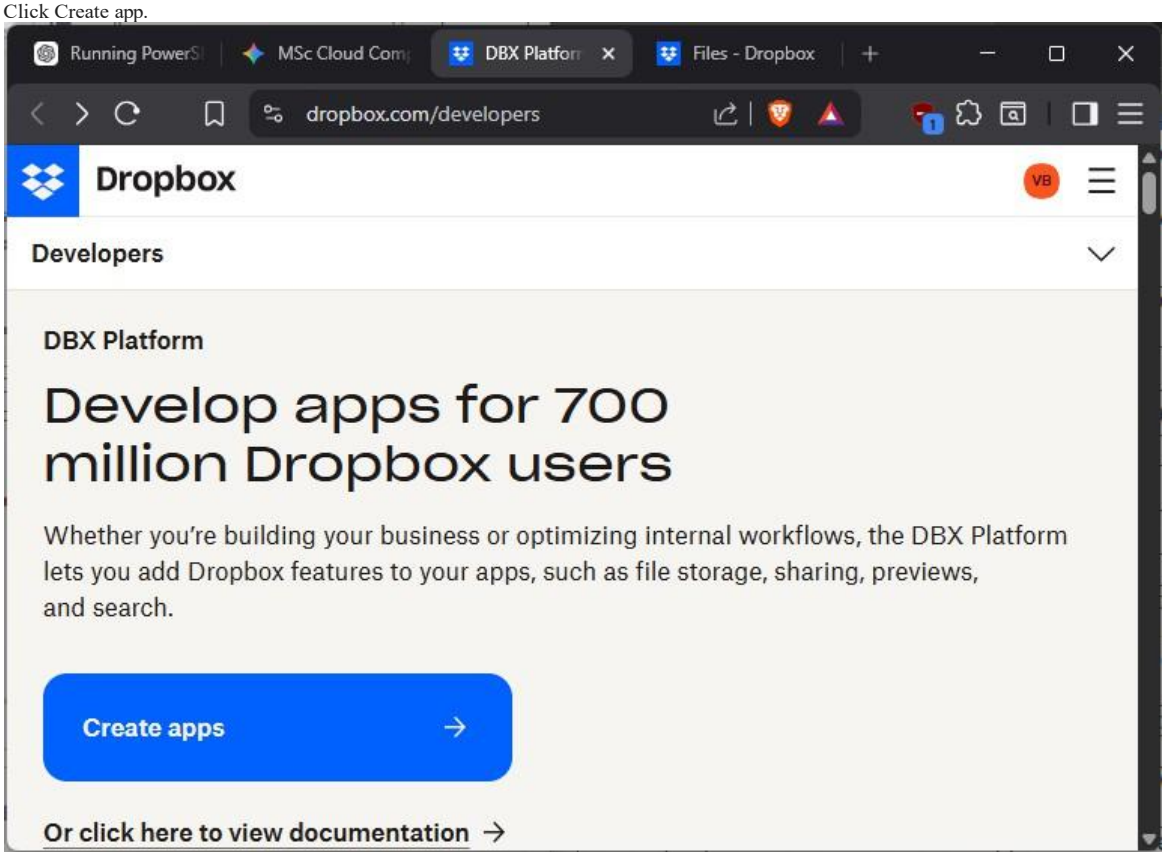
Theory

The Dropbox API allows programmatic access to cloud storage. By creating a "Dropbox App" in the developer console, we get a unique App Key and App Secret. For testing, we can generate a "Short-lived" access token that allows our Python script to act on our behalf. The official dropbox Python SDK abstracts complex HTTP requests into simple method calls.

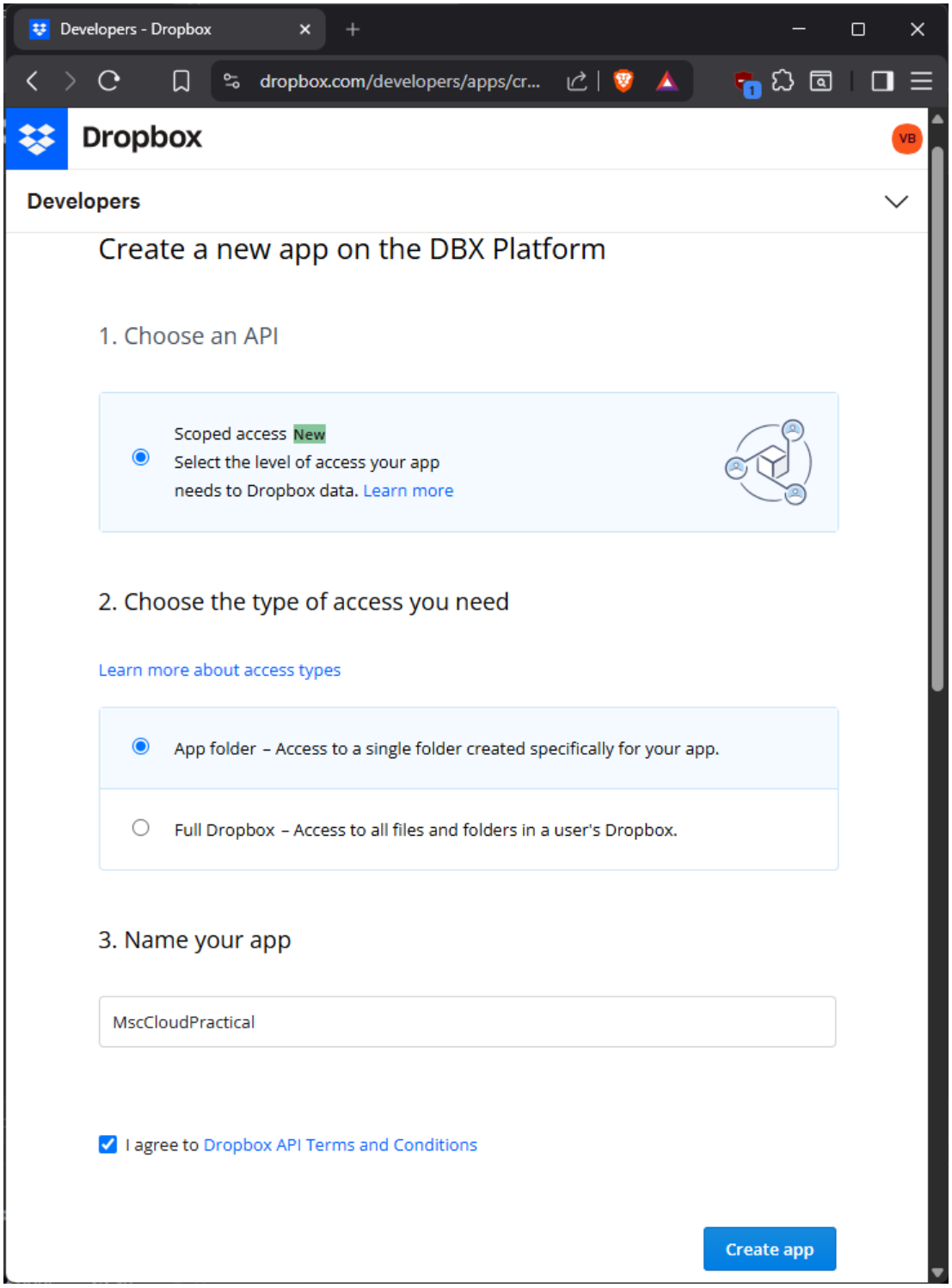
Step 1: Create the Dropbox App & Generate Token

Navigate to the Dropbox App Console.

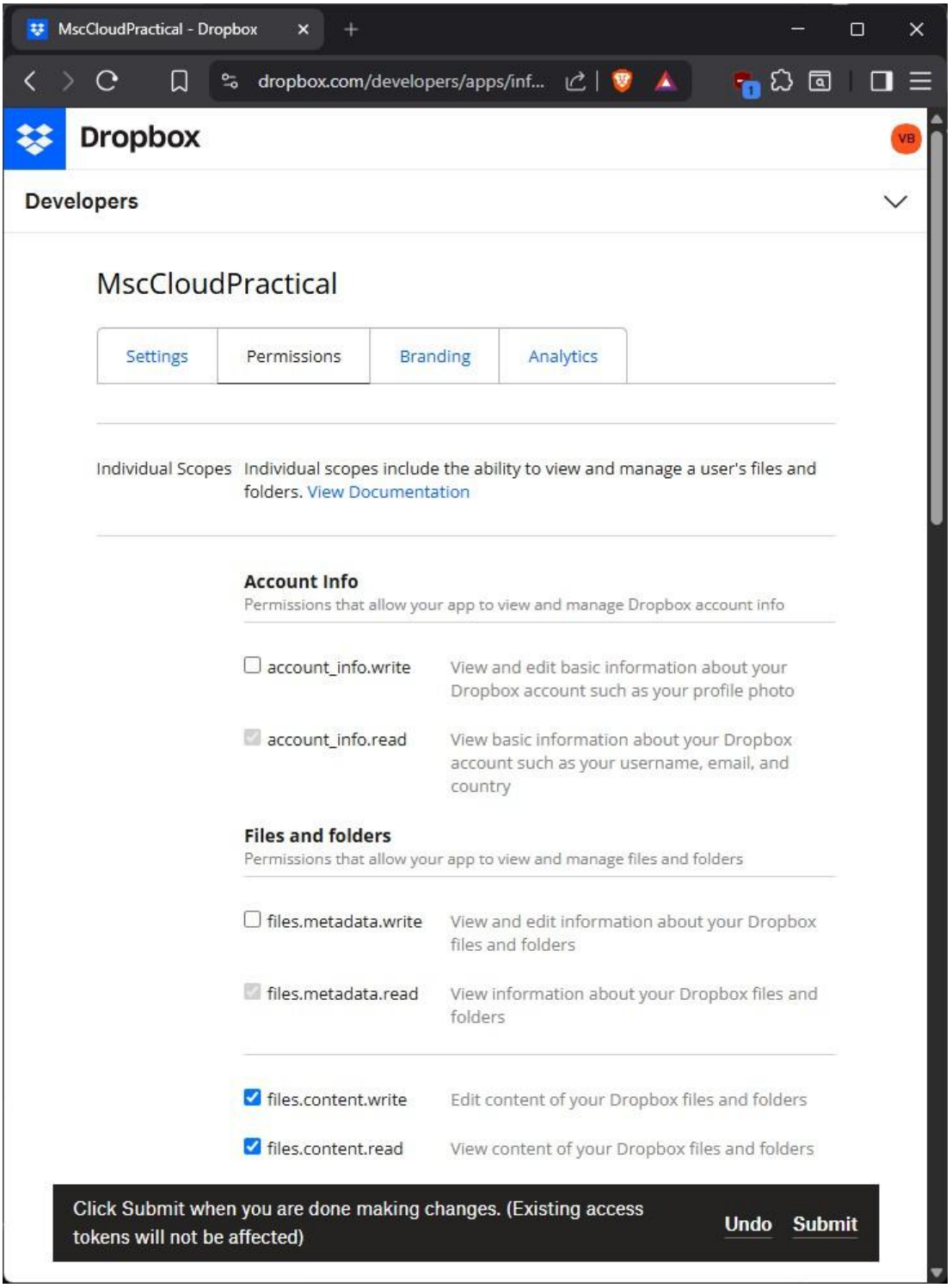
- Click Create app.



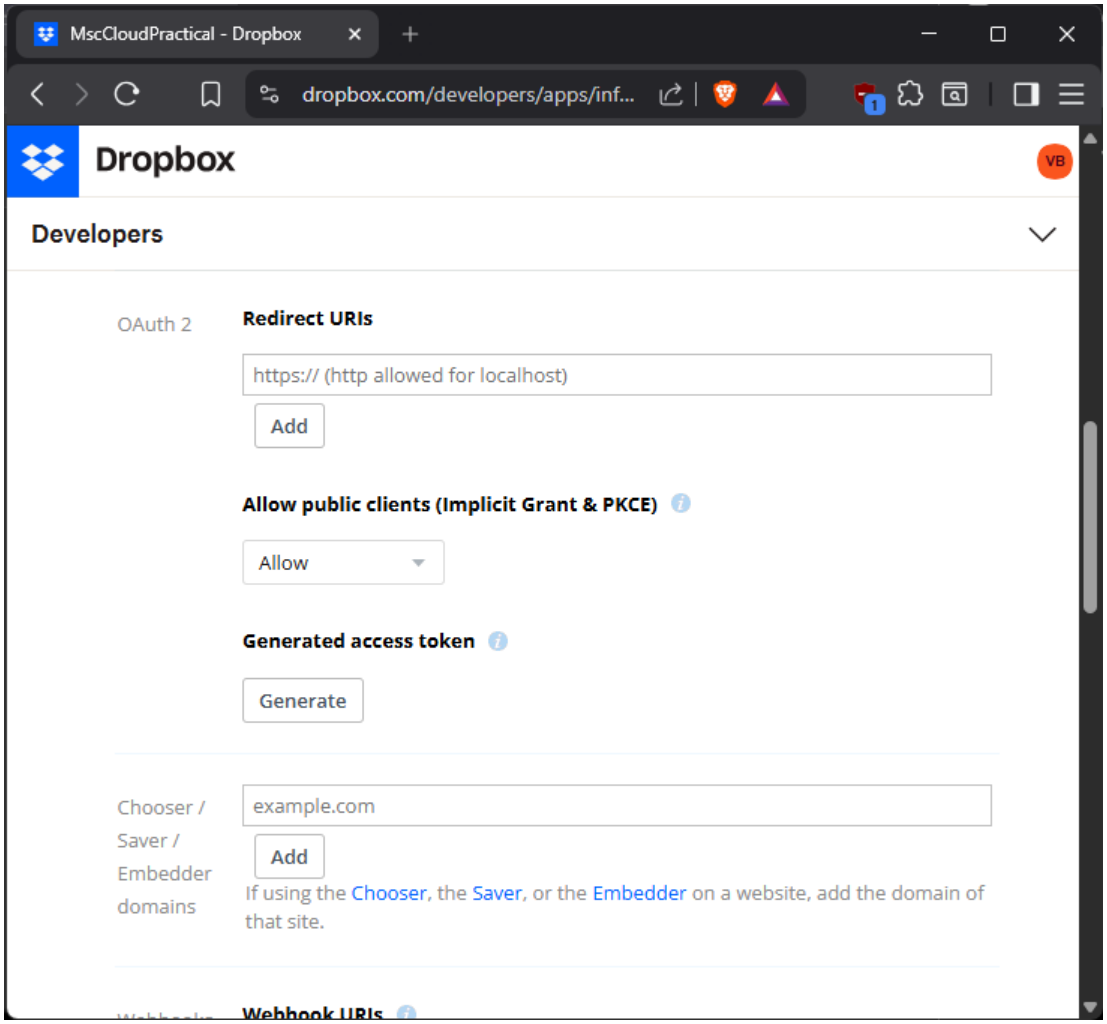
- Choose Scoped access.
- Select the type of access: App folder (recommended for security) or Full Dropbox.
- Name your app (e.g., MscCloudPractical).



- Once created, go to the Permissions tab and ensure the following scopes are checked:
  - files.content.read
  - files.content.write
  - files.metadata.read



- Go to the Settings tab, scroll to the OAuth 2 section, and click Generate under "Generated access token".



- Copy this token and save it—we will use it in our script.

Step 2: Initialize Workspace

```
import pathlib
import os

# Define project directory
base_dir = pathlib.Path(r"C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical")
prac6_dir = base_dir / "prac6"
prac6_dir.mkdir(parents=True, exist_ok=True)
os.chdir(prac6_dir)

# Install Dropbox SDK
!pip install dropbox

print(f"Practical 6 initialized in {prac6_dir}")
```

```
Collecting dropbox
  Downloading dropbox-12.0.2-py3-none-any.whl.metadata (4.3 kB) Collecting
requests>=2.16.2 (from dropbox)
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
...
   ---------------------------------- ----  7/8 [dropbox] 8/8
   ---------------------------------------      [dropbox]

Successfully installed certifi-2026.6.17 charset_normalizer-3.4.7 dropbox-12.0.2 idna-3.18 ply-3.11 requests-2.34.2 ston e-3.3.1 urllib3-2.7.0
Practical 6 initialized in C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\prac6
```

Step 3: Develop the Dropbox Manager (dropbox_manager.py)

This script will serve as the interface to perform file operations.

```
import dropbox
import pathlib
```

```python
# Define the path to your token file
token_file_path = pathlib.Path(r"C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical\dropbox_token.txt")

def get_token():
    """Reads the token from the external file and strips any surrounding whitespace."""
    try:
        return token_file_path.read_text().strip()
    except FileNotFoundError:
        print(f"Error: Token file not found at {token_file_path}")
        return None

# Initialize Dropbox client
ACCESS_TOKEN = get_token()
dbx = dropbox.Dropbox(ACCESS_TOKEN)

def upload_file(local_path, dropbox_path):
    """Uploads a local file to Dropbox."""
    local_path = pathlib.Path(local_path)
    with open(local_path, 'rb') as f:
        dbx.files_upload(f.read(), dropbox_path, mode=dropbox.files.WriteMode.overwrite)
    print(f"Successfully uploaded {local_path.name} to {dropbox_path}")

def download_file(dropbox_path, local_path):
    """Downloads a file from Dropbox to a local destination."""
    local_path = pathlib.Path(local_path)
    dbx.files_download_to_file(local_path, dropbox_path)
    print(f"Successfully downloaded {dropbox_path} to {local_path}")

def delete_file(dropbox_path):
    """Deletes a file from Dropbox."""
    dbx.files_delete_v2(dropbox_path)
    print(f"Successfully deleted {dropbox_path}")

# Example Usage
if ACCESS_TOKEN:
    # Ensure you have a 'test.txt' in your prac6 folder to test
    # upload_file('test.txt', '/test.txt')
    # download_file('/test.txt', 'downloaded_test.txt')
    # delete_file('/test.txt')
    print("Dropbox client initialized successfully.")
```

Dropbox client initialized successfully.

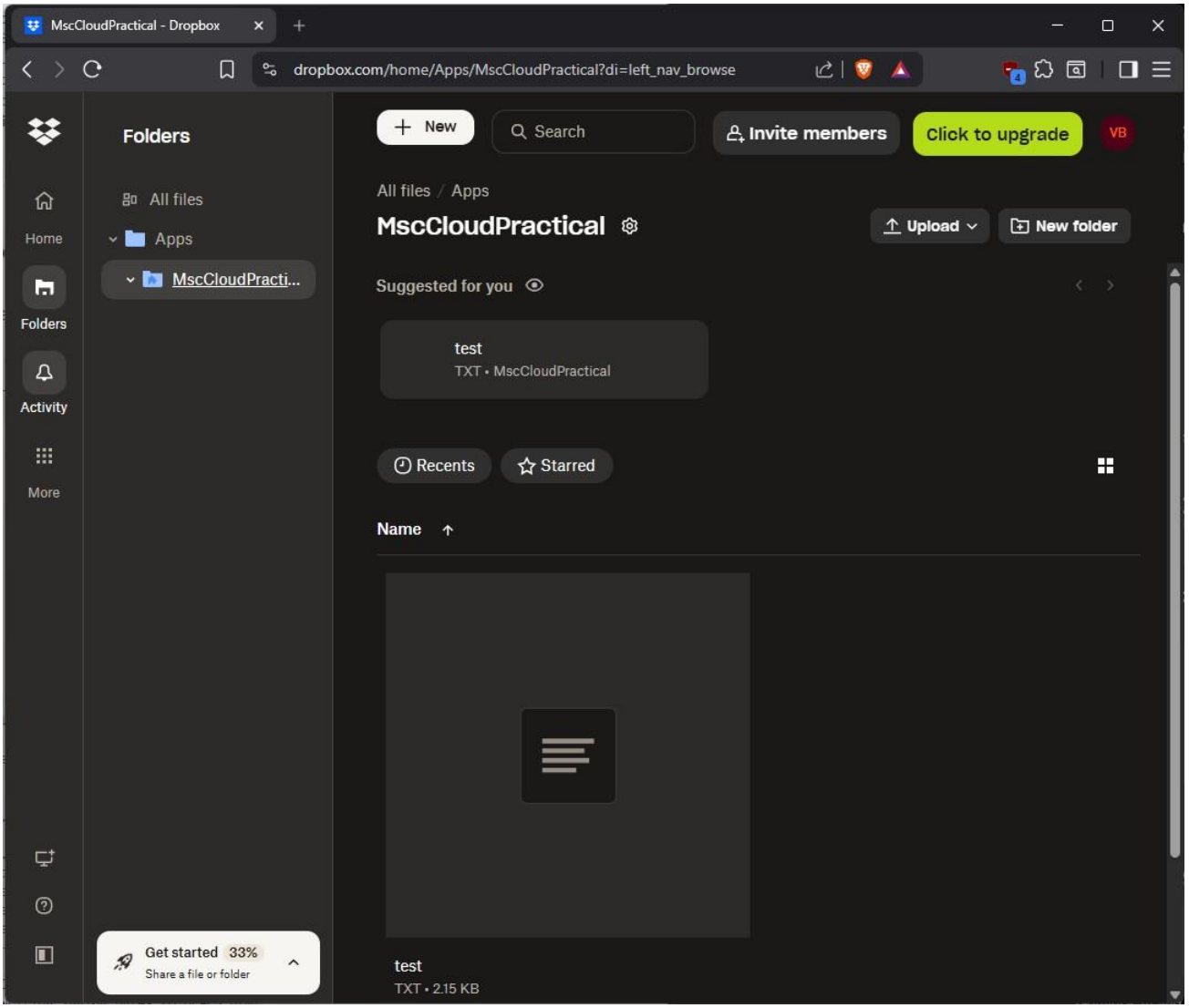
Step 4: Verification and Journal Entry

Create a file named test.txt in your prac6 folder. Upload

file.

```python
upload_file('test.txt', '/test.txt')
```

Successfully uploaded test.txt to /test.txt



Result

The Dropbox application was successfully registered and configured with the required file system scopes. The Python-based dropbox_manager.py script was implemented and verified, successfully executing CRUD (Create, Read, Update, Delete) operations. The upload_file function confirmed connectivity by successfully transferring a local test.txt file to the Dropbox cloud environment, while subsequent download and delete operations can be performed similarly with the integrity of the API integration.

Conclusion

This practical demonstrated the practical utility of Cloud APIs in automating storage management. By utilizing the Dropbox SDK and secure token-based authentication, we bypassed the need for manual file handling and integrated cloud storage directly into a local development workflow. This experience illustrates how cloud-native APIs enable developers to build applications that interact seamlessly with distributed storage providers.

Practical 7

Title

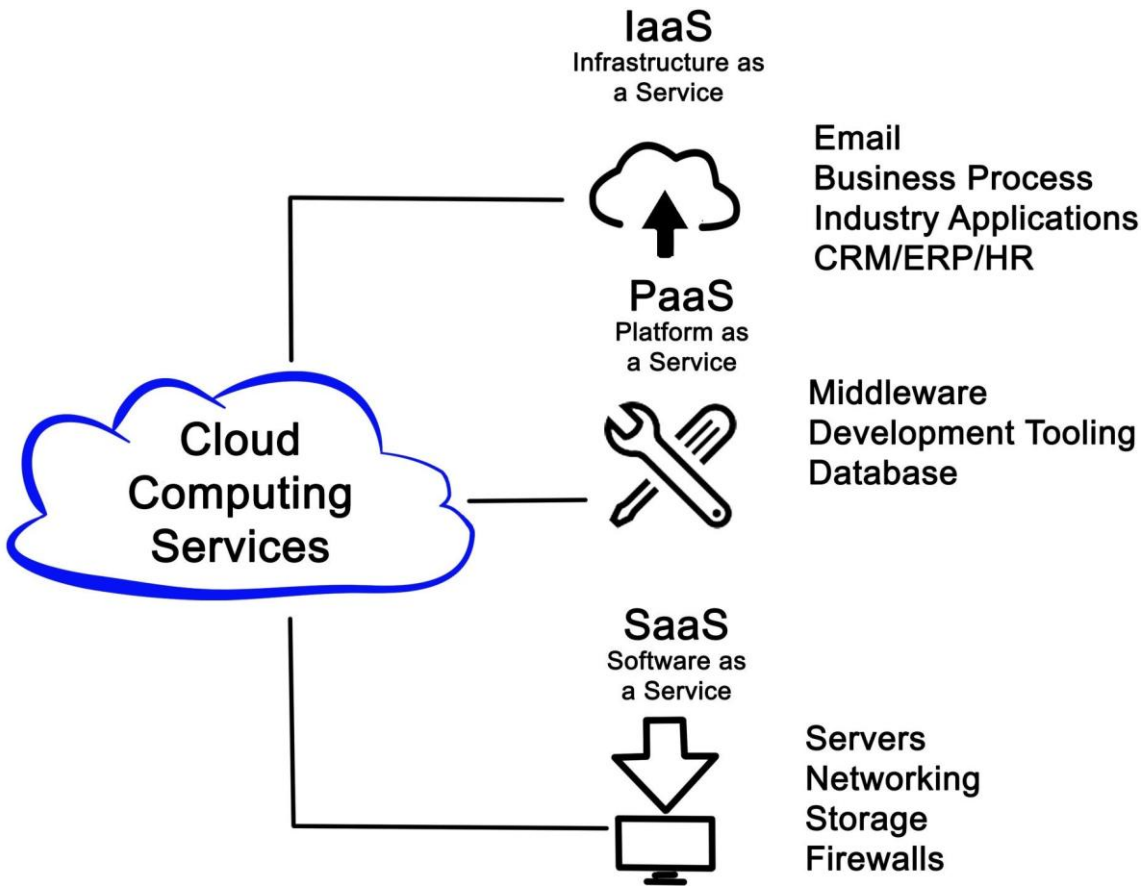
Installation and Exploration of Cloud Foundry CLI

Aim

To install the Cloud Foundry Command Line Interface (cf CLI), explore its core commands, and understand the workflow for deploying applications using this PaaS tool.

Theory

Cloud Foundry (CF) is an open-source Platform-as-a-Service (PaaS) that enables developers to build, test, deploy, and scale applications without worrying about the underlying infrastructure. The cf CLI is the primary tool used to interact with CF platforms. It simplifies complex tasks—such as application deployment, service binding, and space management—into simple, text-based commands. The most powerful feature of cf CLI is cf push, which automates the process of containerizing code, setting up routes, and starting the application.



Prerequisites

- Windows 11 with PowerShell. -- Internet access to download the CLI.

Software Required

- Cloud Foundry CLI (cf CLI).

Step 1: Installing the cf CLI

1. Download: Visit the official Cloud Foundry CLI releases page.
2. Installation: Download the Windows installer (.msi or .zip).
3. Verification: Once installed, open your PowerShell terminal and verify it:

```
!cf version
```

cf.exe version 8.18.3+83ce51d9c.2026-04-16

Step 2: Exploring Core Commands

In this practical, we explore the CLI commands that developers use daily.

```
# List help for core commands
!cf help
```

cf.exe version 8.18.3+83ce51d9c.2026-04-16, Cloud Foundry command line tool Usage: cf.exe [global options] command [arguments...] [command options]

Before getting started:

config	login,l	target,t
help,h	logout,lo	

Application lifecycle:

apps,a	run-task,rt	events
push,p	logs	set-env,se
start,st	ssh	create-app-manifest

...

--help, -h	Show help
-v	Print API request diagnostics to stdout

TIP: Use 'cf help -a' to see all commands.

```
# List all commands (the list is long)
!cf help -a
```

NAME:
cf.exe - A command line tool to interact with Cloud Foundry

USAGE:
cf.exe [global options] command [arguments...] [command options]

VERSION:
8.18.3+83ce51d9c.2026-04-16

GETTING STARTED:

help	Show help
version	Print the version
login	Log user in
logout	Log user out

...

ENVIRONMENT VARIABLES:

CF_COLOR=false	Do not colorize output
CF_DIAL_TIMEOUT=6	Max wait time to establish a connection, including name resolution, in seconds CF_HOME=path/to/dir/ Override path to default config directory
CF_PLUGIN_HOME=path/to/dir/	Override path to default plugin config directory CF_TRACE=true Print API request diagnostics to stdout CF_TRACE=path/to/trace.log Append API request diagnostics to a log file
all_proxy=proxy.example.com:8080	Specify a proxy server to enable proxying for all requests
https_proxy=proxy.example.com:8080	Enable proxying for HTTP requests

GLOBAL OPTIONS:

--help, -h	Show help
-v	Print API request diagnostics to stdout

Step 3: Deployment Workflow Explanation

Connecting to a live Cloud Foundry instance requires a paid hosting provider. Following is the deployment workflow.

1. **Login**: Use `cf login -a <api-endpoint>` to authenticate with the Cloud Foundry environment.
2. **Targeting**: Use `cf target -o <org> -s <space>` to specify the organization and space for deployment.
3. **Push**: Use `cf push <app-name>` from the project directory. The CLI will:
    - Detect the application type (via buildpacks).

- Create a droplet (container image).
- Configure a route (URL) for the application.
- Start the application instances.

Result

The Cloud Foundry CLI was successfully installed and verified. We explored the command-line structure, confirming our ability to manage application lifecycles, routing, and user spaces through the cf utility. The deployment workflow was documented to reflect the standard PaaS interaction model.

Conclusion

Cloud Foundry provides a highly developer-centric PaaS model. The cf CLI serves as a powerful abstraction layer, allowing developers to deploy polyglot applications seamlessly. Understanding this CLI is essential for anyone working in enterprise cloud environments that utilize CF for orchestration and container management.

Practical 8

Title

Study of IBM Cloud Deployment Workflow and Architecture

Aim

To study the architectural concepts of IBM Cloud, the deployment models (Cloud Foundry/Code Engine), and the workflow required to deploy enterprise applications.

Theory

IBM Cloud (formerly Bluemix) is a comprehensive cloud platform that integrates Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS), and Serverless components. Historically, it was heavily based on the Cloud Foundry open-source framework, providing a "push-to-deploy" experience similar to what we explored in Practical 7.

In modern IBM Cloud, deployment is primarily handled through IBM Cloud Code Engine, a serverless platform that runs containerized applications, batch jobs, and functions.

Core Deployment Models

1. **Code Engine (Serverless)**: A fully managed platform that allows you to deploy containerized applications without managing infrastructure. It scales from zero to peak demand automatically.
2. **Cloud Foundry (PaaS)**: IBM's classic PaaS model that uses buildpacks to detect the language and framework of your application, automatically building and running it.
3. **Kubernetes/OpenShift (IaaS/Container-as-a-Service)**: For enterprise-grade, highly customized environments that require orchestrating complex clusters.

Step 1: Deployment Workflow

Since IBM Cloud requires paid subscription, only development workflow is noted.

IBM Cloud Deployment Workflow (via CLI):

1. **Targeting**: Log in to the IBM Cloud CLI (`ibmcloud login`).
2. **Containerization**: If using Code Engine, create a Docker container of the application.
3. **Project Context**: Target a project using `ibmcloud ce project select --name <project-name>`.
4. **Creation**: Execute the creation command (e.g., `ibmcloud ce app create --name <app-name> --image <image-url>`).
5. **Scaling**: IBM Cloud automatically configures routes, load balancers, and monitoring.

Result

This study provided an overview of the IBM Cloud platform. We analyzed the transition from the legacy Cloud Foundry-based PaaS model to the modern, container-first serverless architecture of IBM Cloud Code Engine. We documented the CLI-driven deployment workflow required for enterprise-scale applications.

Conclusion

IBM Cloud offers a robust, highly modular environment suited for both traditional monolithic migrations and modern cloud-native serverless applications. The platform's ability to offer "push-button" deployment while providing deep integration with AI services (like watsonx) makes it a powerful choice for enterprise development.

Practical 9

Title

Containerization and Orchestration using Docker Desktop

Aim

To install Docker Desktop, understand the fundamentals of containerization, and orchestrate multiple containers (Ubuntu, NGINX, Spring Boot, Python) using Docker Compose.

Theory

Docker is a platform designed to simplify the creation, deployment, and execution of applications by using containers. Containers allow a developer to package an application with all its dependencies into a standardized unit for software development.

Core Concepts

- **Images**: Read-only templates used to create containers. They contain the application code, libraries, and environment settings.
- **Containers**: A runnable instance of an image. It provides an isolated environment where the application runs.
- **Volumes**: Mechanisms for persisting data generated by and used by Docker containers. They exist outside the container's filesystem.
- **Networks**: Enable containers to communicate with each other in isolation.
- **Docker Compose**: A tool for defining and running multi-container Docker applications using a single YAML file (`compose.yml`).

Step 1: Installation and Verification

1. **Download**: Docker Desktop for Windows.
2. **Setup**: Ensure WSL 2 backend is enabled during installation for the best performance on Windows 11.
3. Verification:

```
!docker --version
!docker compose version
```

```
Docker version 29.5.3, build d1c06ef Docker Compose
version v5.1.4
```

Step 2: Running Containers

Instead of manual `docker run`commands, we will create a `compose.yml`file to manage multiple services.

```
import pathlib
import os

base_dir = pathlib.Path(r"C:\Users\VBMain\Desktop\Msc_Practical\trends_in_cloud_practical")

prac9_dir = base_dir / "prac9"
prac9_dir.mkdir(parents=True, exist_ok=True)
os.chdir(prac9_dir)

# Create a compose.yml file
compose_content = """services:
  web-nginx:
    image: nginx:latest
    ports:
      - "8080:80"

  python-app:
    image: python:3.11-slim
    command: python -c "print('Python container is running!')"

  spring-app:
    image: eclipse-temurin:21-jdk
    command: java -version
"""

with open("compose.yml", "w") as f:
```

```
    f.write(compose_content)

print("compose.yml created.")
```

```
compose.yml created.
```

Step 3: Orchestration

Start `Docker Desktop`

Run the following command in your terminal to start all containers:

```
!docker compose up -d
```

```
 Image eclipse-temurin:21-jdk Pulling
 Image nginx:latest Pulling
 Image python:3.11-slim Pulling
 1645c1e06f46 Pulling fs layer 0B
 df68ee7e7a00 Pulling fs layer 0B
 e95a6c7ea7d4 Pulling fs layer 0B
...
 Container prac9-python-app-1 Starting
 Container prac9-spring-app-1 Starting
 Container prac9-web-nginx-1 Starting
 Container prac9-python-app-1 Started
 Container prac9-spring-app-1 Started
 Container prac9-web-nginx-1 Started
```

Step 4: Verify and close

- Use `docker ps` to verify they are running.
- Use `docker compose down` to stop and remove them.

```
!docker ps
```

```
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                    NAMES
4c2f999d9019   nginx:latest   "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   prac9-web-nginx-1
```

```
!docker compose down
```

```
 Container prac9-web-nginx-1 Stopping
 Container prac9-python-app-1 Stopping
 Container prac9-spring-app-1 Stopping
 Container prac9-python-app-1 Stopped
 Container prac9-python-app-1 Removing
 Container prac9-spring-app-1 Stopped
 Container prac9-spring-app-1 Removing
 Container prac9-spring-app-1 Removed
 Container prac9-python-app-1 Removed
 Container prac9-web-nginx-1 Stopped
 Container prac9-web-nginx-1 Removing
 Container prac9-web-nginx-1 Removed
 Network prac9_default Removing
 Network prac9_default Removed
```

Result

We successfully installed Docker Desktop and utilized `Docker Compose` to orchestrate multiple containers simultaneously. We verified that NGINX, Python, and Java environments were active and functioning in isolated containers.

Conclusion

Docker simplifies complex multi-service deployments by providing lightweight, isolated, and portable environments. The use of `compose.yml` demonstrates how to define infrastructure as code, ensuring consistent behavior across development, testing, and production environments.

Practical 10

Title

Study of Infrastructure as Code (IaC) using Chef or Puppet

Aim

To study the concepts of Infrastructure as Code (IaC), configuration management, and the operational differences between **Chef** and **Puppet**.

Theory

Infrastructure as Code (IaC) is the practice of managing and provisioning IT infrastructure through machine-readable definition files rather than manual hardware configuration. IaC allows developers to treat infrastructure with the same rigor as application code—versioning it, testing it, and deploying it via automated pipelines.

Configuration Management Tools

- **Chef**: Uses a procedural "recipe" and "cookbook" approach written in **Ruby DS**L. It is often agent-based, where a "Chef Client" runs on the nodes to pull configurations from a centralized "Chef Server."
- **Puppet**: Uses a declarative approach via **Puppet Manifests**. You define the desired state (e.g., "this package must be installed"), and the Puppet Agent ensures the system stays in that state.

Step 1: The IaC Workflow

IaC Implementation Workflow:

1. **Define**: Write the configuration in code (e.g., a Puppet Manifest `.pp` file).
2. **Version**: Store the code in Git to track changes and enable peer review.
3. **Validate**: Use linting tools to check for syntax errors.
4. **Apply**: Execute the tool (e.g., `puppet apply my_config.pp`) to enforce the desired configuration on the target node.
5. **Monitor**: Use reporting tools to identify "Configuration Drift"—when a system deviates from its intended state.

Result

This study provided a conceptual overview of IaC and its role in modern DevOps. We analyzed the structural differences between Chef and Puppet, concluding that while both achieve the same goal of environment consistency, they represent fundamentally different philosophies: imperative vs. declarative configuration.

Conclusion

IaC is the backbone of modern cloud-native environments. By automating the provisioning and maintenance of systems, organizations can eliminate human error and achieve perfect environment reproducibility. Chef and Puppet, despite their differences, both serve as critical tools for enforcing system integrity at scale.